

Trabalho Prático 1: Treehouses

Resolução de Problemas com Grafos - Unidade 3

Pedro Kauã Castro
Eduardo Suaki
Robson Santos

Universidade de Fortaleza (UNIFOR)
Prof. Me Ricardo Carubbi

25 de maio de 2026

- **Objetivo:** Conectar todas as casas na árvore com o menor custo de cabos possível.
- **O Desafio (Variação):** Algumas casas já estão previamente conectadas (custo zero).
- **Exigência Técnica:** Alta precisão no cálculo de ponto flutuante (erro tolerado menor que 0.001).

- **Vértices (V):** Cada casa na árvore representa um vértice, definido por coordenadas (X, Y) no plano bidimensional.
- **Arestas (E):** Possíveis ligações entre as casas. Forma-se um grafo implícito **completo**, pois qualquer casa pode se conectar a qualquer outra.
- **Pesos:** O peso de cada aresta é a **distância euclidiana** entre as coordenadas.
- **Alvo:** Encontrar a Árvore Geradora Mínima (MST) com **componentes pré-conectados**.

Algoritmo Escolhido: Kruskal

Passos da Solução:

- 1 Ler as coordenadas de todas as casas.
- 2 Unir imediatamente as conexões pré-existentes (custo zero).
- 3 Calcular a distância euclidiana de todos os pares que ainda não estão conectados.
- 4 Ordenar essas novas arestas pelo menor peso.
- 5 Aplicar o algoritmo de Kruskal para finalizar a rede.

O Papel do Union-Find (DSU)

A estrutura DSU é o coração da estratégia, com duas funções centrais:

- **Inicialização (union):** Antes de calcular as distâncias, agrupa as conexões garantidas pelo enunciado, formando “ilhas” consolidadas.
- **Prevenção de Ciclos (find):** Durante a execução do Kruskal, checka se as extremidades do cabo já fazem parte da mesma rede.
 - Se *sim*: descarta a aresta.
 - Se *não*: aplica o union e soma a distância ao custo total.

- **Complexidade de Tempo Total:** $O(V^2 \log V)$
 - *Criação das arestas:* $O(V^2)$
 - *Ordenação (Etapa Dominante):* $O(E \log E)$. Como o grafo é quase completo ($E \approx V^2$), temos $O(V^2 \log V)$.
 - *Kruskal + DSU:* $O(E \cdot \alpha(V))$, onde α é o inverso de Ackermann (quase constante).
- **Complexidade de Espaço (Memória):** $O(V^2)$
 - Necessário para armazenar a lista de todas as arestas possíveis geradas antes de rodar o Kruskal.

- **Múltiplas “Ilhas”:** O grafo inicial pode apresentar vários componentes desconexos. A ordenação global das arestas restantes e o Kruskal garantem a conexão final de todos eles.
- **Ausência de conexões extras ($p = 0$):** Validação implementada para não quebrar a leitura da entrada caso não existam cabos adicionais iniciais.
- **Precisão Numérica:** Uso rigoroso de formatação (`Locale.US, %.6f`) no Java para evitar vereditos de *Wrong Answer* gerados por arredondamento indevido.

Obrigado!

Alguma dúvida?